

Data Structures and Algorithms

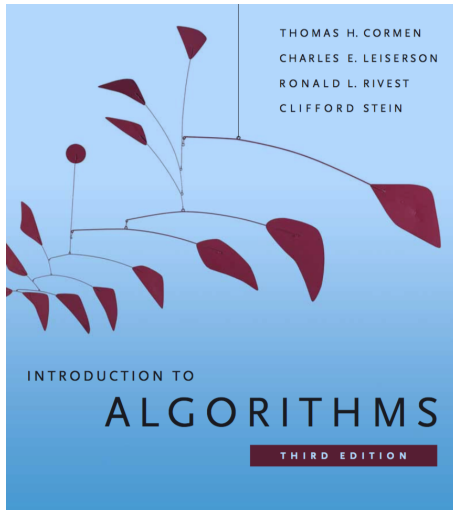
GÉZA HORVÁTH

First Class

Lecture topics

- 1 Introductory examples of the design and analysis of algorithms.
- 2 Asymptotic behavior of functions, asymptotic notation, further examples of simple algorithms.
- 3 Basic notions concerning data structures: modeling, abstraction. Set, multiset, array, matrix.
- 4 Elementary data structures: stacks, queues, lists.
- 5 Tables, hash tables, hash functions, collisions.
- 6 Trees, binary trees, binary search trees, traversal, searching, insertion, deletion.
- 7 Balanced binary search trees, AVL trees.
- 8 Red-black trees.
- 9 B-trees.
- 10 Graphs, traversal, shortest path.
- 11 Parallel algorithms: matrix multiplication, merge sort.
- 12 Decision problems, the complexity classes P and NP.
- 13 Summary.

References: lecture notes + book



1+2+...+100 standard method with for loop

Output:

$$x = \sum_{i=1}^{100} i$$

Pseudocode:

ADDING-NUMBERS()

- ❶ x=0
- ❷ for i=1 to 100
- ❸ x=x+i
- ❹ return x

C code:

```
#include <stdio.h>
int sum(){
    int i,x;
    x=0;
    for(i=1;i<=100;i++)
        x=x+i;
    return x;
}
int main(){
    printf("%d",sum());
}
```

Pseudocode conventions

ADDING-NUMBERS()

- ❶ $x=0$
 - ❷ for $i=1$ to 100
 - ❸ $x=x+i$
 - ❹ return x
- Indentation indicates block structure. For example, the body of the for loop that begins on line 2 consists of line 3 only.
 - Looping constructs: for, while, and repeat-until.
 - Conditional construct: if-else.
 - The symbol " //" indicates that the remainder of the line is a comment.
 - Details: book pages 20-22 "Pseudocode conventions". We are going to follow these conventions (more or less).

1+2+...+100 standard method with for loop

Output:

$$x = \sum_{i=1}^{100} i$$

Pseudocode:

ADDING-NUMBERS()

- ① $x=0$
- ② for $i=1$ to 100
- ③ $x=x+i$
- ④ return x

Number of steps: 100

1+2+...+n standard method with for loop

Input: $n \geq 1$

Output:

$$x = \sum_{i=1}^n i$$

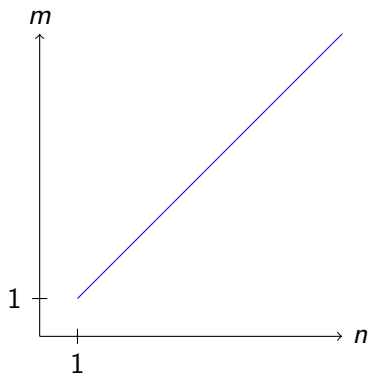
Pseudocode:

ADDING-NUMBERS(n)

- 1 $x=0$
- 2 for $i=1$ to n
- 3 $x=x+i$
- 4 return x

Number of steps: n

$1+2+\dots+n$ standard method with for loop



n – input size

m – running time

Input size

Input size – "The best notion for input size depends on the problem being studied. For many problems, such as sorting or computing discrete Fourier transforms, the most natural measure is the number of items in the input – for example, the array size n for sorting. For many other problems, such as multiplying two integers, the best measure of input size is the total number of bits needed to represent the input in ordinary binary notation. Sometimes, it is more appropriate to describe the size of the input with two numbers rather than one. For instance, if the input to an algorithm is a graph, the input size can be described by the numbers of vertices and edges in the graph. We shall indicate which input size measure is being used with each problem we study." (Book page 25.)

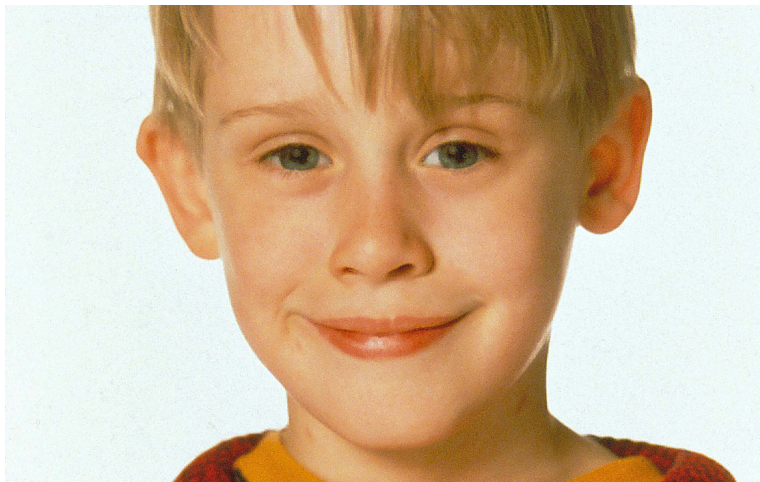
Running time

Running time – "The running time of an algorithm on a particular input is the number of primitive operations or "steps" executed. It is convenient to define the notion of step so that it is as machine-independent as possible. For the moment, let us adopt the following view. A constant amount of time is required to execute each line of our pseudocode. One line may take a different amount of time than another line, but we shall assume that each execution of the i th line takes time c_i , where c_i is a constant." (Book page 25.)

Carl Friedrich Gauss (1777-1855)



Once he was a boy...



1+2+...+100 with the idea of Gauss

1	2	3	4	5	...	46	47	48	49	50
+	+	+	+	+	...	+	+	+	+	+
100	99	98	97	96	...	55	54	53	52	51
=	=	=	=	=	...	=	=	=	=	=
101	101	101	101	101	...	101	101	101	101	101

1+2+...+100 with the formula of Gauss

1	2	3	4	5	...	46	47	48	49	50
+	+	+	+	+	...	+	+	+	+	+
100	99	98	97	96	...	55	54	53	52	51
=	=	=	=	=	...	=	=	=	=	=
101	101	101	101	101	...	101	101	101	101	101

$$x = \sum_{i=1}^{100} i = 50 * 101$$

1+2+...+n with the formula of Gauss

1	2	3	4	5	...
+	+	+	+	+	...
n	n-1	n-2	n-3	n-4	...
=	=	=	=	=	...
n+1	n+1	n+1	n+1	n+1	...

$$x = \sum_{i=1}^n i = \frac{n}{2} * (n + 1) = \frac{n * (n + 1)}{2}$$

1+2+...+n with the formula of Gauss

Input: $n \geq 1$

Output:

$$x = \sum_{i=1}^n i$$

Pseudocode:

ADDING-NUMBERS(n)

① return $n*(n+1)/2$

C code:

```
#include <stdio.h>
int sum(int n){
    return n*(n+1)/2;
}
int main(){
    printf("%d",sum(100));
}
```


1+2+...+n with the formula of Gauss

Input: $n \geq 1$

Output:

$$x = \sum_{i=1}^n i$$

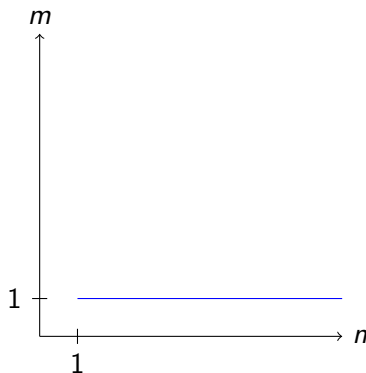
Pseudocode:

ADDING-NUMBERS(n)

① return $n*(n+1)/2$

Number of steps: 1

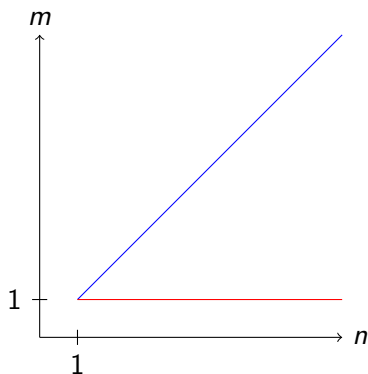
$1+2+\dots+n$ with the formula of Gauss



n – input size

m – running time

$1+2+\dots+n$ comparison



n – input size

m – running time

Factorial standard algorithm

Input: $n \geq 1$

Output: $x = n!$

Pseudocode:

FACTORIAL(n)

- ① $x=1$
- ② for $i=2$ to n
- ③ $x=x*i$
- ④ return x

Number of steps: n

C code:

```
#include <stdio.h>
int factorial(int n){
    int i,x;
    x=1;
    for(i=2;i<=n;i++)
        x=x*i;
    return x;
}
int main(){
    printf("%d",factorial(5));
}
```

Factorial recursive algorithm

Input: $n \geq 1$

Output: $x = n!$

Pseudocode:

FACTORIAL(n)

- ① if $n=1$
- ② return 1
- ③ else
- ④ return $n * \text{FACTORIAL}(n-1)$

Number of steps: n

C code:

```
#include <stdio.h>
int factorial(int n){
    return n==1?1:n*factorial(n-1);
}
int main(){
    printf("%d",factorial(5));
}
```

Linear search

Input: vector $A[n]$, number x

Output: the position of x in A

Pseudocode: practice

Number of steps: ???

C code:

```
#include <stdio.h>
int linear_search(int A[], int n, int x){
    int i;
    for(i=0;i<n;i++){
        if(A[i]==x)
            return i;
    }
    return -1;
}

int main(){
    int A[5]={1,2,3,4,5};
    printf("%d",linear_search(A,5,3));
}
```

Linear search

Input: vector $A[n]$, number x

Output: the position of x in A

Pseudocode: practice

Number of steps:

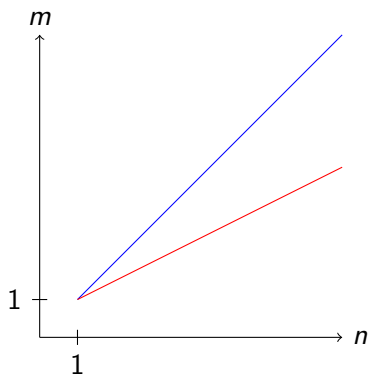
- best-case: nobody cares
- average-case: $(n+1)/2$
- worst-case: n

C code:

```
#include <stdio.h>
int linear_search(int A[], int n, int x){
    int i;
    for(i=0;i<n;i++){
        if(A[i]==x)
            return i;
    }
    return -1;
}

int main(){
    int A[5]={1,2,3,4,5};
    printf("%d",linear_search(A,5,3));
}
```

Linear search



n – input size

m – running time

Binary search

Input: vector $A[n]$, number x

Output: the position of x in A

Pseudocode: practice

C code: practice

Number of steps:

- worst-case: ???

Binary search

Input: vector $A[n]$, number x

Output: the position of x in A

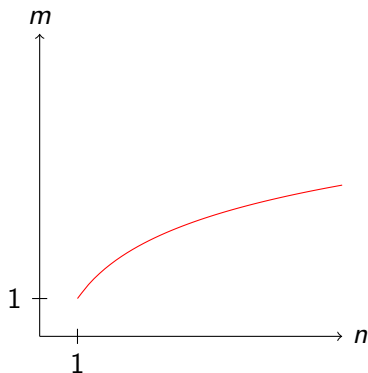
Pseudocode: practice

C code: practice

Number of steps:

- worst-case: $\log_2(n)+1$

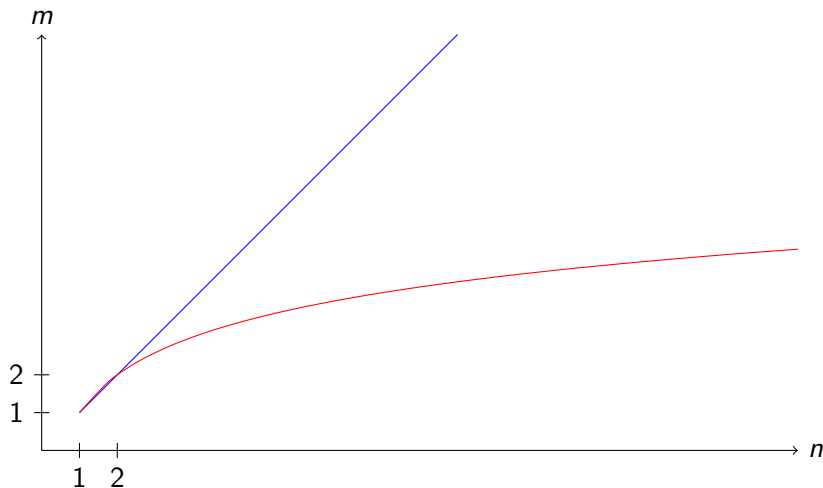
Binary search



n – input size

m – running time

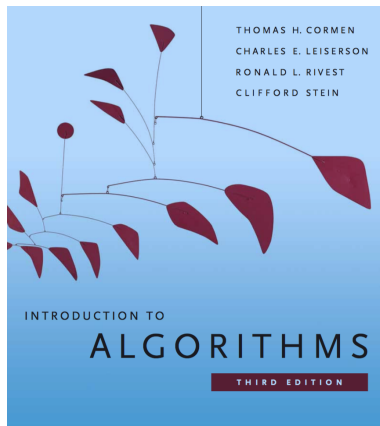
Linear and binary search comparison



n – input size

m – running time

References



This work was supported by the construction EFOP-3.4.3-16-2016-00021. The project was supported by the European Union, co-financed by the European Social Fund.